

Memory Layer

Concept Design Document

Tenant-Scoped Continuity, Recall, and Auditable History Across the Agent Platform

Fortune 500 and Regulated Enterprises	SaaS Subscription and Usage	Phase 2 Concept
Target Market	Model	Stage

Contents

Executive Summary

Aligns with: Architecture, the Memory Row across Layers 1 to 3

The Secure, Orchestration, and Intelligence layers each answer a question about a single unit of work: is it safe, does it move, does it decide correctly. None of them answers a question that holds across all work, over time: what has happened before, what was decided, and can it be recalled and reconstructed when it matters. Today that concern is not absent; it is scattered. Worker storage holds profiles, the Intelligence layer grounds against historical decisions, orchestration holds thread state, and the Secure layer retains audit events. The same concept lives in four places under four names.

The Memory Layer is not a fourth layer in the stack. It is a horizontal capability that crosses every functional layer, a row over the columns, giving each layer a disciplined, consistent way to record and recall long-term history while preserving the tenant isolation the platform is built on. It converts four scattered fragments into one contract.

What the Memory Layer Delivers

- Recall by reference: every unit of work carries a handle that resolves to relevant prior context (decisions, rationales, and history) without copying data forward.
- Tenant-scoped continuity: per-tenant memory that accumulates a tenant's own decision history, fully isolated, inside the tenant's own account boundary.
- Auditable reconstruction: any decision can be replayed against the exact profile, runtime version, policy, and model that produced it.
- Two-bodies governance: a clean separation between what the platform operator must prove and what the tenant must prove, linked only by reference.
- Reusable platform knowledge: de-identified operational patterns that compound across the install base: patterns, never raw tenant data.

One line: Orchestration moves work and Intelligence decides it. Memory is how the platform remembers what it did: correctly scoped, fully isolated, and reconstructable under audit.

The Problem

Aligns with: Production Failure Modes, Memory as an Undefined Row

A platform can isolate, orchestrate, and reason correctly and still fail in production because memory is treated as an afterthought distributed across other concerns. The failure is not loud; it is a slow erosion of the properties regulated buyers require.

Failure Mode	Consequence
Memory smeared across layers with no shared contract	Four inconsistent implementations; no single place to reason about recall, retention, or isolation
Recall by data copy instead of reference	Stale duplicated context, drift from the source of truth, and uncontrolled spread of sensitive data
No tenant boundary on accumulated history	Cross-tenant exposure of decisions and protected data: the single most damaging failure for a regulated platform
History not pinned to the version that produced it	Audit replay reconstructs decisions against the wrong runtime; the replay guarantee becomes aspirational
Immutability and deletion rights left unreconciled	Append-only audit needs collide with right-to-deletion and retention limits, surfacing late in a customer's compliance review
One combined vault for all memory shapes	High-frequency events, aggregates, and knowledge forced through a single substrate that fits none of them well

Regulated deployments require not just that work happened, but that it can be recalled, attributed, and reconstructed, a bar that scattered, ad hoc state cannot meet.

Product Vision

Aligns with: Product Overview, A Row Over the Columns

The Memory Layer is a reusable continuity-and-recall substrate. It is not a new control plane, not a replacement for orchestration state or grounding, and not a single combined vault. It is the discipline and the interface that let every layer record and recall long-term history the same way, behind one contract, without ever crossing the tenant boundary.

The Core Reframe: Row, Not Column

The built layers decompose the platform by function. Each slices what the system does to a single request. Memory does not slice a request; it is a property that must hold across all requests, over time. Modeled as a matrix, the functional layers are columns and memory is a row that spans them. A row does not need its own column; it needs a cell in each column and one contract those cells all satisfy.

Column (Functional Layer)	Memory Cell in That Column	What the Row Contract Enforces
Secure	Audit-event retention and security history	Append-only, attributed, tenant-scoped, reconstructable
Orchestration	Checkpoint and long-run workflow / thread state	Same lifecycle, same isolation, same interface shape
Intelligence	Grounded recall over historical decisions and rationale	Recall by reference; relevance scoped to the tenant

Recall is by reference, not by store. The platform already persists run specs, decision artifacts, explainability packs, audit events, and checkpoints. The Memory Layer does not duplicate that data; it makes it findable. Every unit of work carries a reference marker, built on the correlation identifiers the platform already threads through every event, that resolves to relevant prior context only when a step actually requests it. The marker is cheap to carry; the dereference happens on demand.

The most important property of a reference is whether it still resolves. The Memory Layer's central contract is the resolution rule: when a marker cannot resolve, the platform must treat it as an explicit, audited condition, never a silent absence, because in a platform whose audit replay is a named guarantee, a quietly dangling reference is invisible until the one moment it matters.

Memory Composition: Three Substrates, One Interface

Aligns with: Architecture, Substrate Selection by Data Shape

“Memory” bundles three data shapes that want different storage. The Memory Layer presents them through one retrieval interface, so the layers above never need to know which substrate served an answer. Behind that interface, each shape lives where it belongs. Forcing all three through a single vault because they share a name is a category error.

Memory Shape	What It Holds	Natural Substrate	Why
Episodic history	Ordered record of what happened, when, in what sequence	Event store / append-only log	Sequential writes at throughput; ordering from the event's own clock, not arrival time
Aggregate views	Distributions and statistics computed over episodic history	Metrics / derived store or materialized views	Numerical aggregation; rebuildable by replaying events; never scanned from notes
Knowledge and rationale	Decisions, entities, relationships, the “why” behind outcomes	Linked knowledge vault (Obsidian-style)	Human-inspectable, portable, graph-walkable; links map to entity-and-relationship recall

The vault metaphor fits exactly one shape. A markdown-and-links vault is the right choice for knowledge and rationale, where portability, diffability, and a navigable graph are genuine advantages. It is the wrong choice for episodic events at volume and for aggregate computation. The Memory Layer therefore specifies a vault for the knowledge cell and purpose-built stores for the other two: unified at the interface, distinct at the substrate.

The relevance index is existing machinery, not new infrastructure. “Does this relate to anything seen before” is a retrieval query, not a cache. It is the same embeddings-and-tenant-scoped-index capability the Intelligence layer already specifies for Domain RAG, pointed inward at internal history instead of outward at external corpora. This collapses the most infrastructure-heavy-looking part of memory into a capability already on the roadmap.

Vault Separation: Platform Vault and Tenant Vault

Aligns with: Trust Boundaries, Two Vaults on Opposite Sides of the Tenant Line

There are two vaults, and they share nothing but the word. They sit on opposite sides of the tenant boundary that the entire Secure layer exists to enforce, and they differ on every axis that matters. Treating them as one segment would reintroduce, one level up, the exact isolation risk the platform is designed to eliminate.

Axis	Platform Vault (“Tenant Zero”)	Tenant Vault (per tenant, ×N)
Owner	Platform operator	The individual tenant
Holds	Operational knowledge: remediation playbooks, reusable patterns, platform-health and architecture history	Private workflow memory: decisions, grounded history, rationale, claim and clinical context
Tenant scope	None by construction: zero tenant payload, zero PHI	Exactly one, hard-enforced; PHI-bearing
Location	Shared control plane / Shared Services OU, beside the SVA and log archive	Inside the tenant’s own account (Data account) in the micro-account topology
Readers	Operators, the operational observer, platform governance	That tenant’s agents and authorized humans only, not the operator
Linked-graph model	Permitted: no tenant data, so cross-linking is a feature to keep	Permitted within the tenant; forbidden across tenants
Lifecycle	Persists with the platform	Dies with the account at offboarding (quarantine + close)

The boundary rule, stated flatly: a tenant vault never reads another tenant vault, and the platform vault never holds anything that could reconstruct a single tenant’s protected data. This rule protects the architecture twice: it keeps tenants isolated, and it keeps the platform vault clean enough that the convenient linked-graph model remains safe to use.

The Platform–Tenant Flow

Aligns with: Cross-Tenant Pattern Reuse, Patterns Not Raw Data

The two vaults have one relationship: directional, governed, and deliberately lossy. Getting the direction and the loss right is the whole game, and it is the same de-identification discipline the platform already requires before any training use of protected data.

Tenant → Platform: De-identified Patterns Only

The platform vault learns from what happens in tenant workloads; that is how playbooks and reusable patterns accumulate. What crosses is the de-identified pattern, never the raw instance. The shape of a problem and its resolution can promote; the claim, the patient, the tenant identity, and the underlying data cannot. This is an extraction that strips everything identifying on the way out, not a sync, replication, or mirror.

Platform → Tenant: Advisory Pattern, Grounded Locally

When the operational observer meets a situation in a tenant and queries the platform vault for prior resolutions, what returns is a de-identified pattern. That pattern is then applied within the tenant's own boundary and grounded against the tenant's own vault and data. The pattern is shared; the grounding is always local. This delivers the flywheel (memory makes the next resolution faster) without ever carrying one tenant's specifics into another.

The moment anything tenant-identifying lands in the platform vault, two things break at once: tenant isolation, and the property that lets the platform vault use the linked-graph model in the first place. The de-identification rule is therefore not a courtesy; it is what keeps both the isolation guarantee and the platform vault's own simplicity intact.

Memory and Governance: The Two-Bodies Seam

Aligns with: Security and Compliance, Independent Governing Bodies

Governance is not a layer or a component; it is a discipline applied to every layer's significant events: emit an immutable, attributed record to the appropriate ledger. In a platform consumed by regulated tenants, there are two distinct governing bodies, each accountable to its own regulators for different things. Neither can discharge the other's obligation.

	Platform Operator Governance	Tenant Governance
Governs	That the platform behaved correctly: isolation held, gates fired, credentials were scoped and expired, the validation pipeline ran	That decisions were correct and authorized: grounded, properly approved, human-reviewed where required, reconstructable
Audience	The operator's own auditor (SOC 2-class), and enterprise security diligence on the platform as a vendor	The tenant's regulator (HIPAA and equivalents) and the tenant's own compliance function
Jurisdiction	The control plane and the boundaries between tenants	The content of the tenant's workflow
Data	Provably contains no tenant data; proves the mechanism worked without seeing what flowed through it	PHI-bearing; lives and dies inside the tenant account

The records are non-substitutable. The operator's clean record does nothing for the tenant's HIPAA audit, and the tenant's decision record does nothing for the operator's SOC 2. Each governance record must be complete and verifiable on its own side of the tenant boundary, by a reader with no access to the other side.

The only seam is a reference. A de-identified correlation handle (run or version identifier) appears in both records. The operator's record says a run was gate-enforced at a time with a proof hash; the tenant's record says the same run was approved by these roles against this rationale. Joining the two requires both parties to cooperate, which is the correct property: reconstructing the full picture should require mutual consent, not be silently doable by either alone.

Break-Glass: Two Unrelated Processes, One Shared Floor

Tenant break-glass is the tenant troubleshooting its own workflow inside its own account, entirely tenant-governed; the operator has no part and, by the isolation model, no access. Operator break-glass is the operator working on platform infrastructure, entirely operator-governed and infrastructure-related. The two are unrelated as governance events, coupled only at the physical substrate beneath both.

The one real seam is impact, not access. Operator infrastructure action can affect a tenant's running workloads (a node drain, emergency maintenance) without ever touching tenant data. That is not a confidentiality event but it can be an availability one. It warrants a tenant-facing operational signal at the SLA / notification layer, distinct from governance, so a tenant can correlate "our workload was interrupted and we didn't do it" against the shared floor moving.

Version Pinning as Governed Workflow State

Aligns with: Audit Replay, Runtime Version as First-Class Provenance

The runtime version a tenant is running is treated as governed workflow state (pinned, recorded per run, and replay-critical) exactly as the agent profile version already is. This single decision makes version awareness structural rather than bolted on, and it is delivered through a publish / subscribe model with tenant-controlled adoption.

Event	Governing Body	Meaning
Publish	Platform operator	Version N+1 is built, scanned, signed, eval-passed, and made available. Changes nothing running in any tenant.
Adopt	Tenant	The tenant elects to consume a version, at a time, under its own authority. Recorded in the tenant's own provenance.

It is always both events, split across the publish/adopt seam. The class of change sets the constraints on each side, not whether both occur.

Change Class	Adoption Policy	Governance Note
Feature / capability	Pull: tenant chooses when	Routine; tenant-governed adoption
Security fix	Push with deadline: operator may compel within a window	Forced adoption writes to both bodies: the tenant's record marks it as mandated, not elected
Critical	Forced / immediate	Near-joint event; elevated retention on both sides
Deprecation (end-of-life)	Operator push with deadline	Both-bodies event: operator gives notice; tenant records its migration

What pinning buys, by construction. Because every decision artifact carries the version that produced it, audit replay reconstructs against the correct runtime automatically; the adopt event is self-evidencing (runs before the transition carry the old version, runs after carry the new); and the both-bodies seam needs no new plumbing, since the version identifier is the correlation handle already present on both sides. Profile pinning made what the agent was configured to do replayable; runtime pinning makes the machinery that did it replayable. Together, a regulated decision is fully reconstructable from its run.

Memory Lifecycle: Immutability and Deletion

Aligns with: Compliance Posture, Reconciling Append-Only with Right-to-Deletion

Immutability and deletion rights are in direct tension, and memory is where they collide. Governance memory is append-only by design; regulated deployments carry right-to-deletion, retention limits, and legal hold. The resolution is a placement rule rather than a feature.

- Immutable governance memory lives tenant-local and dies with the tenant. Account offboarding (quarantine and close) discharges deletion obligations for everything in the tenant's account, because closing the account removes the data with it.
- Nothing tenant-identifying that must be immutable may live in a shared store, where append-only would collide head-on with “delete this tenant's data.”
- Corrections are appended, never overwritten. A superseded record stays visible with its correction after it; in-place mutation is never used, preserving the audit trail's integrity.

A retention floor follows from pinning. Because runtime versions are replay-critical and adoption is tenant-controlled, the platform may owe replayability of a version long after no tenant is actively running it, for as long as runs that used it remain within their retention period. This is a constraint to honor when deprecation is defined, not a problem to solve today.

The Operational Observer and Memory

Aligns with: Reliability, Out-of-Band Liveness Detection

Operational liveness (detecting that a workflow has gone silent rather than failed or decided wrongly) is neither a security nor a decision-quality concern, and it is not a memory row. It is an observer: a component that stands outside the matrix and watches the cells, in the same architectural seat the Security Validation Agent already occupies (out-of-band, isolated, observing rather than participating).

Its relationship to memory is twofold and reinforcing. The observer reads the platform vault to retrieve prior resolution patterns when it meets a stuck workflow, and it authors high-value memory when a situation is resolved: the hard-won record of how a class of problem was diagnosed and fixed. De-identified, that record promotes to the platform vault and accelerates the next resolution. The observer is one of memory's most important authors, and memory is what makes the observer compound in value.

Memory, the relevance index, the observer, and governance must remain extensions, not a parallel infrastructure build. The discipline that keeps them so: memory rows extend existing columns, the relevance index is Domain RAG pointed inward, the observer reuses the SVA's out-of-band seat, and governance is an event shape rather than a service. The moment any of them needs its own compute plane and control loop, it has forked.

Regulated Operating Model

Aligns with: Security and Compliance Posture, HIPAA and SOC 2-class

Memory is designed so that the properties regulated buyers test for hold by construction, not by convention.

- Tenant memory is PHI-bearing and inherits every Secure-layer control: per-tenant KMS, no cross-tenant edges, de-identification before any training use.
- The platform vault is provably free of tenant payload, which is what permits operator-side governance to prove the mechanism worked without seeing tenant data.
- Audit replay is reconstructable from run identifier plus pinned profile, runtime, policy, and model versions.
- Recall by reference with an explicit resolution rule means a missing reference is an audited condition, never a silent gap.
- Two governing bodies are separately and independently satisfiable, linked only by a consent-gated correlation handle.

Scope and Phasing

Aligns with: Build Stance, Structural Now versus Growth-on-Demand

Consistent with the platform's “extend, do not fork” stance, the Memory Layer is scoped as a thin, structural commitment now, forward-compatible with a richer build only if a real need pulls it there.

Structural and Decided (Now)	Growth on Demand (Deferred)
Recall by reference; the resolution rule	Vault as primary source of truth for all state
Row contract across the three functional columns	Full event-sourcing with state as a pure projection
Three substrates behind one interface	Aggregate / histogram views as a first-class product surface
Platform vault vs. per-tenant vault separation	Cross-tenant pattern marketplace (consented, productized)
Two-bodies governance seam; correlation handle	Autonomous remediation beyond detect / classify / escalate
Version pinning as governed workflow state	Standalone relevance infrastructure (vs. RAG pointed inward)

A reference marker is precisely what a richer memory layer would dereference into, so committing to the thin version now costs nothing later.

Open Items and Decisions Required

Aligns with: Diligence, Deliberate Deferrals

The following are open by design and tracked as decisions, not gaps. Each has a clear shape; none blocks the architecture.

#	Item	Owner
1	Fan-in / join failure policy at the workflow-edge grain (D needs A, B, C; C is terminally dead): per-edge declared outcome vs. global default	Architecture
2	Resolution rule semantics: precise behavior and audit treatment when a reference marker fails to resolve	Platform Eng
3	Deprecation process and the replay retention floor for versions no tenant actively runs	Platform / Compliance
4	Tenant governance-record split: which approval content stays tenant-local vs. the de-identified proof retained operator-side	Architecture / Compliance
5	Whether the memory row has a real Secure-column cell beyond audit retention (security-relevant recall for the observer), or is empty by nature	Security Eng
6	Governed promotion path from operator working patterns to abstracted tenant-facing capability, treated as a first-class platform-change event	Platform / Product

Diligence Appendix

The following materials are available upon request for technical diligence and partner conversations:

- Memory row contract specification: lifecycle, isolation, and interface shape per column
- Vault separation diagram: platform vault and per-tenant vaults with the directional, de-identified flow
- Two-bodies governance map with the correlation-handle seam and joint-event cases
- Version-pinning provenance model and audit-replay reconstruction walkthrough
- Substrate selection rationale: episodic, aggregate, and knowledge stores behind one interface
- Lifecycle and retention matrix: immutability, deletion, and the pinning-driven retention floor

End of Document